WORKING WITH FUNCTIONS

Introduction

- Large programs are often difficult to manage, thus large programs are divided into smaller units known as **functions.**
- It is simply a group of statements under any name i.e. function name and can be invoked (call) from other part of program.
- Take an example of School Management Software, now this software will contain various tasks like Registering student, Fee collection, Library book issue, TC generation, Result Declaration etc. In this case we have to create different functions for each task to manage the software development.

Functions are of mainly two types:

- **Built in functions:** These are the functions that are already defined in programming system and can be called directly by name for use.
- **User defined functions:** These are the functions that are need to be defined before calling and use.

User Defined Functions

- A function is a set of statements that performs a specific task; a common structuring elements that allows you to use a piece of code repeatedly in different part of program. Functions are also known as sub-routine, methods, procedure or subprogram.
- Syntax to create USER DEFINED FUNCTION

```
def function_name([comma separated list of parameters]):
        statements….
        statements….
```

KEYWORD

FUNCTION DEFINITION

Points to remember…

- Keyword **def** marks the start of function header
- Function name must be unique and follows naming rules same as for identifiers
- Function can take arguments. It is optional
- A colon(:) to mark the end of function header
- Function can contains one or more statement to perform specific task
- An optional **return** statement to return a value from the function.
- Function must be **called/invoked** to execute its code

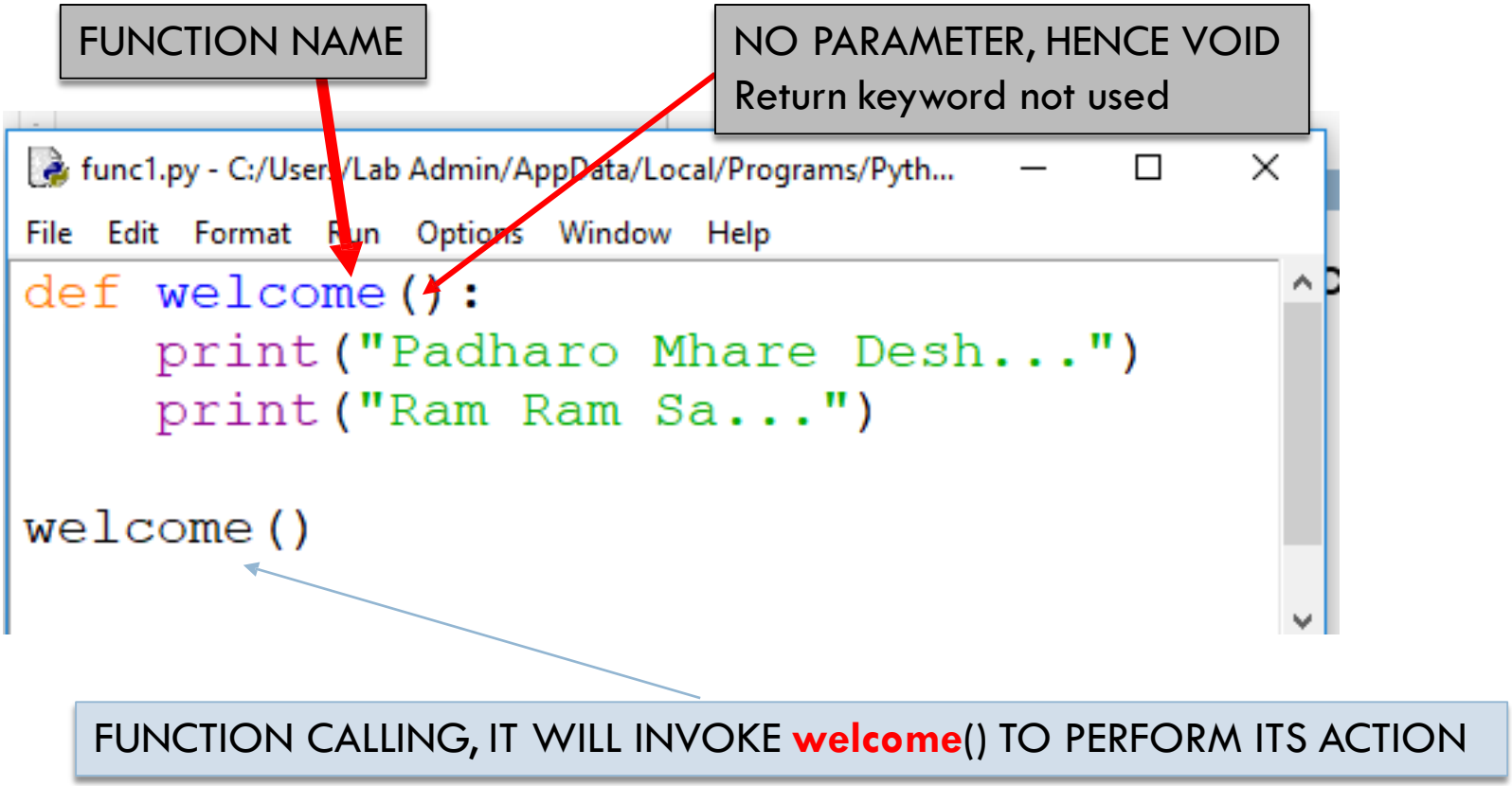
User Defined function can be….

1. Function with no arguments and no return
2. Function with arguments but no return value
3. Function with arguments and return value
4. Function with no argument but return value

Let us understand each of the function type with example….

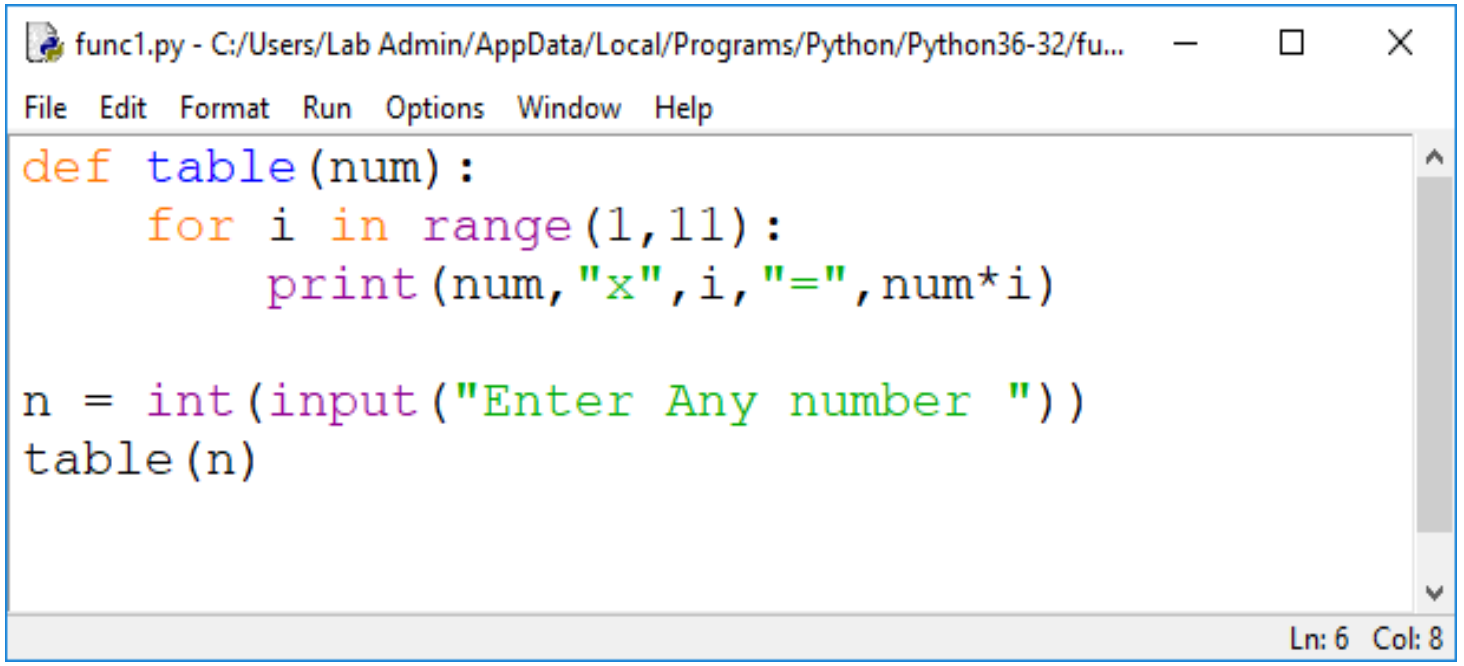
Function with no argument and no return

- This type of function is also known as **void** function



Function with parameters but no return value

- Parameters are given in the parenthesis separated by comma.
- Values are passed for the parameter at the time of function calling.



Function with parameter and return

- We can return values from function using return keyword.
- The return value must be used at the calling place by –
  - Either store it any variable
  - Use with print()
  - Use in any expression

area.py - C:/Users/Lab Admin/AppData/Local/Progra...

File Edit Format Run Options Window Help

```
import math
def area(r):
    a = math.pi * r * r
    return a

n = int(input("Enter Radius "))
ar = area(n)
print("Area of Circle = ",ar)
```

Ln: 8 Col: 0

NOTE: *the return statement ends a function execution even if it is in the middle of function. Anything written below return statement will become unreachable code.*

```
def max(x,y):
        if x>y:
                return x
        else:
                return y
        print("Iam not reachable")
```

Function with no parameter and returning value

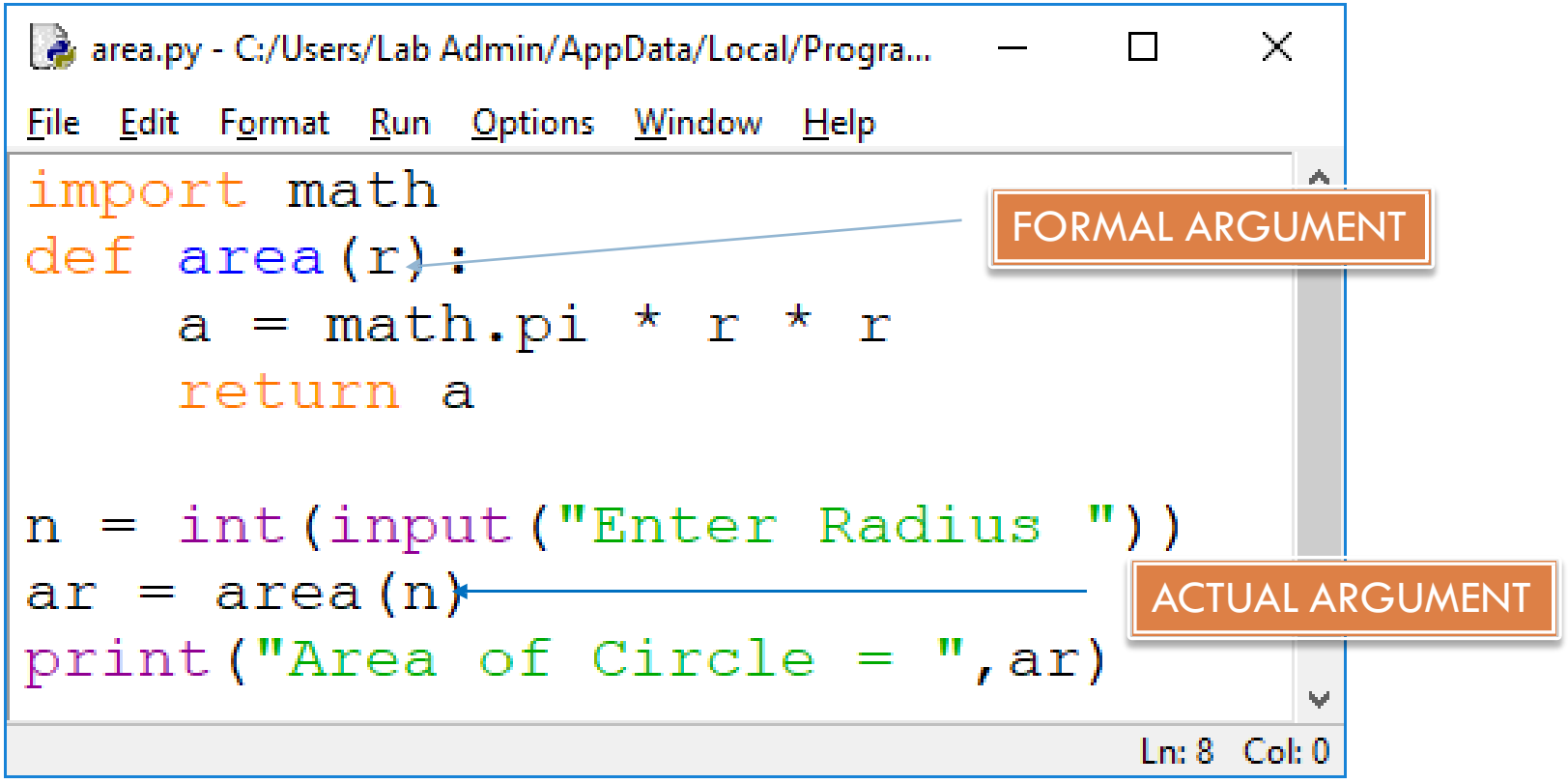
- Function may return a value even without receiving any parameter.

```
def f():
    return "hello"

a=f( )
print(a)
```

```
Output : hello
```

Example of Formal/Actual Arguments



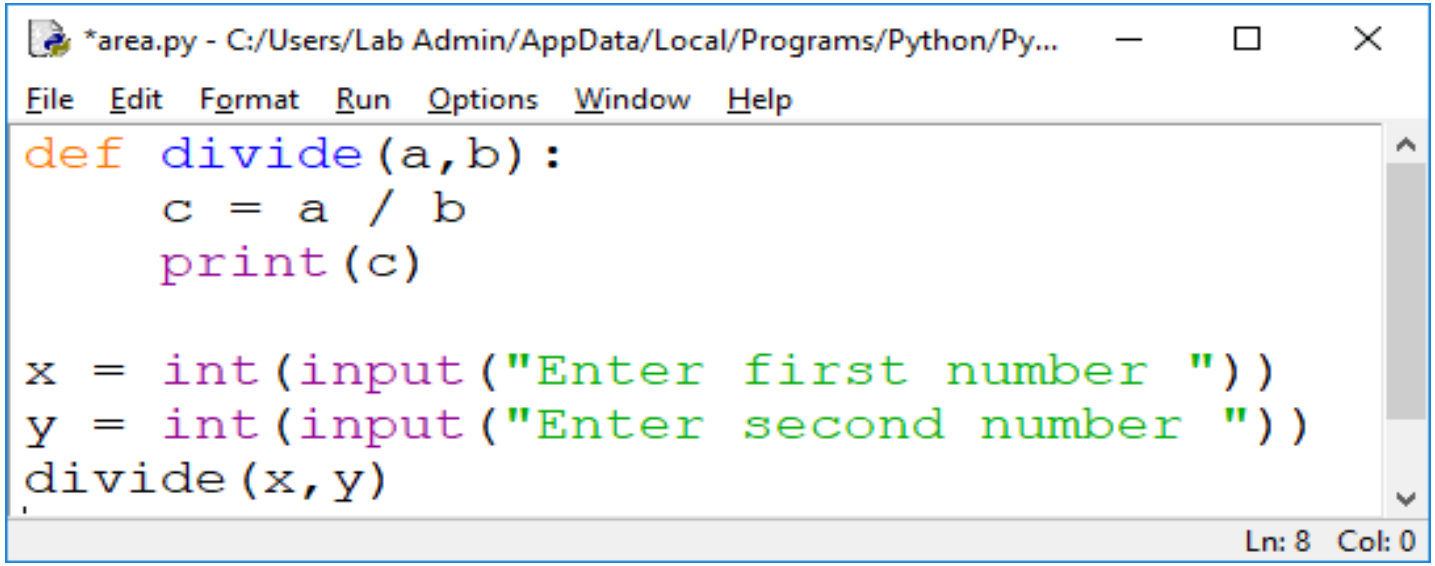
Types of Arguments

- There are various types of Arguments allowed in Python:

1. Positional arguments
2. Default arguments
3. Keyword arguments

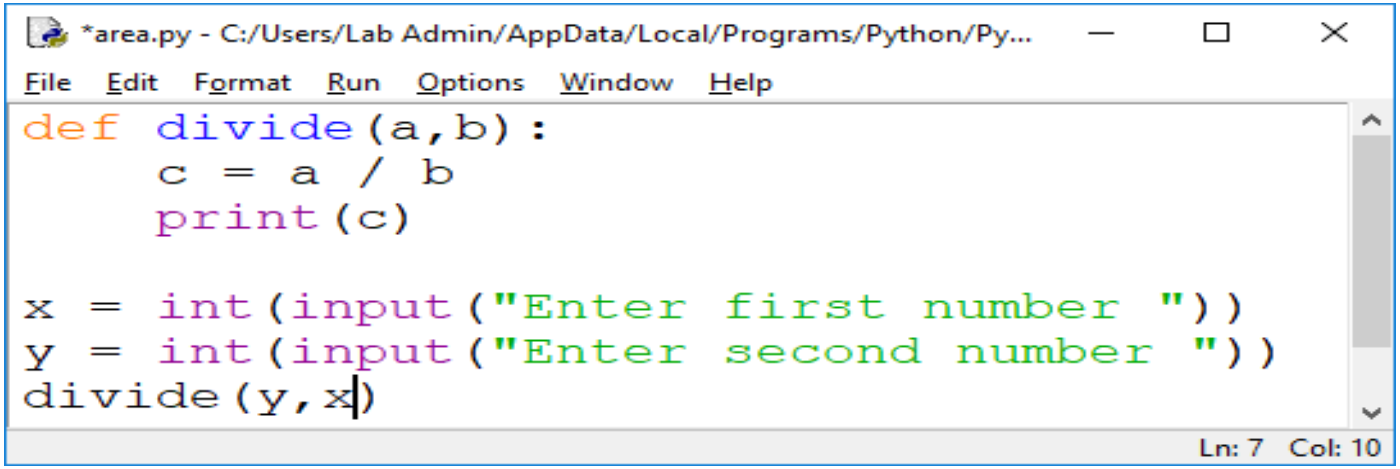
Positional arguments

- Are arguments passed to a function in correct positional order



- Here x is passed to a and y is passed to b i.e. in the order of their position

```
 RESTART: C:/Users/Lab Ad
Enter first number 20
Enter second number 10
2.0
>>>
```




```
 RESTART: C:/Users/Lab Admin/AppData/Local/Progr
Enter first number 20
Enter second number 10
0.5
>>>
```

If the number of formal argument and actual differs then Python will raise an error

Default arguments

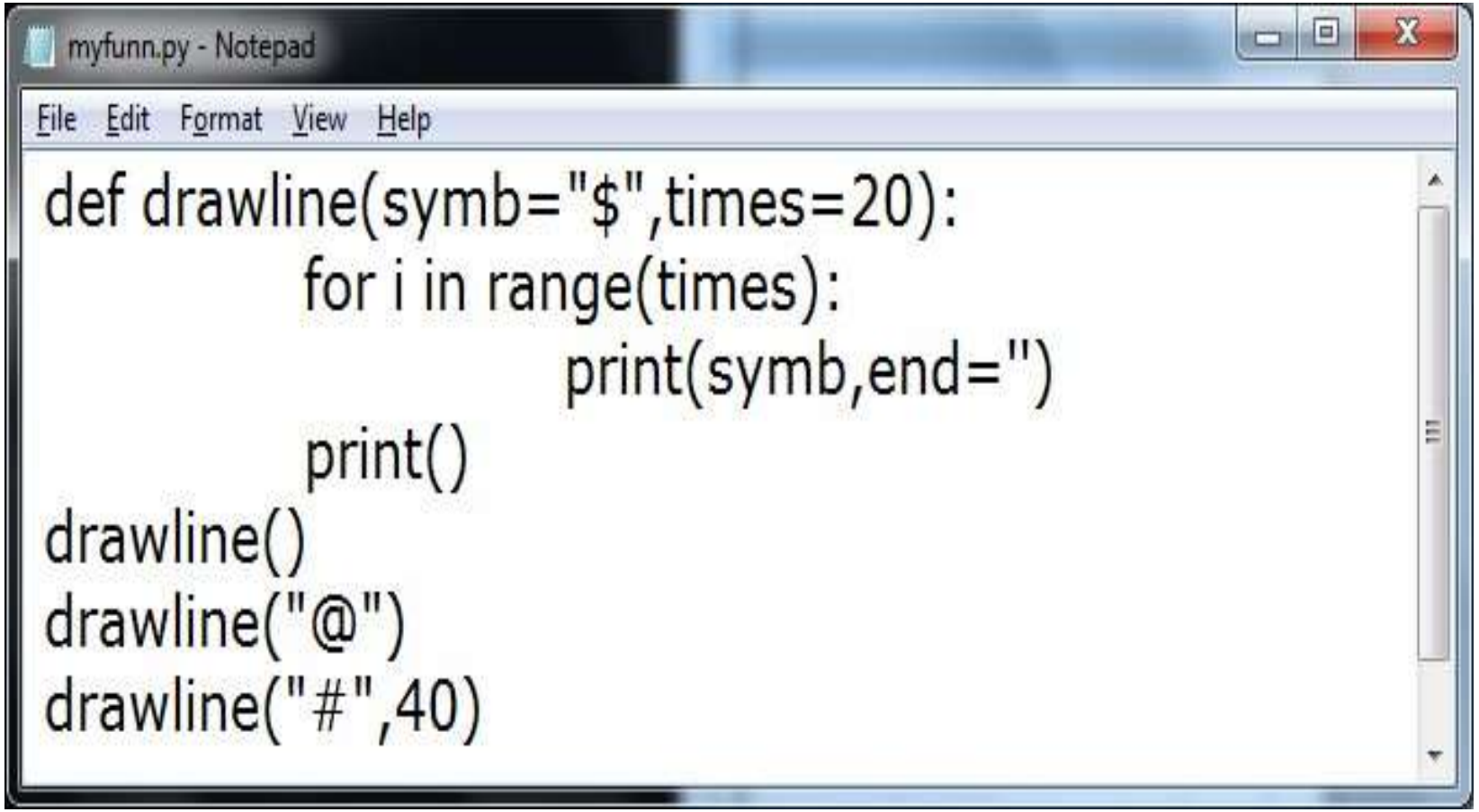
- Sometimes we can provide default values for our positional arguments. In this case if we are not passing any value then default values will be considered.
- Default argument must not followed by non-default arguments.

```
def interest(principal,rate,time=15):
def interest(principal,rate=8.5,time=15):
def interest(principal,rate=8.5,time):
```

VALID

INVALID

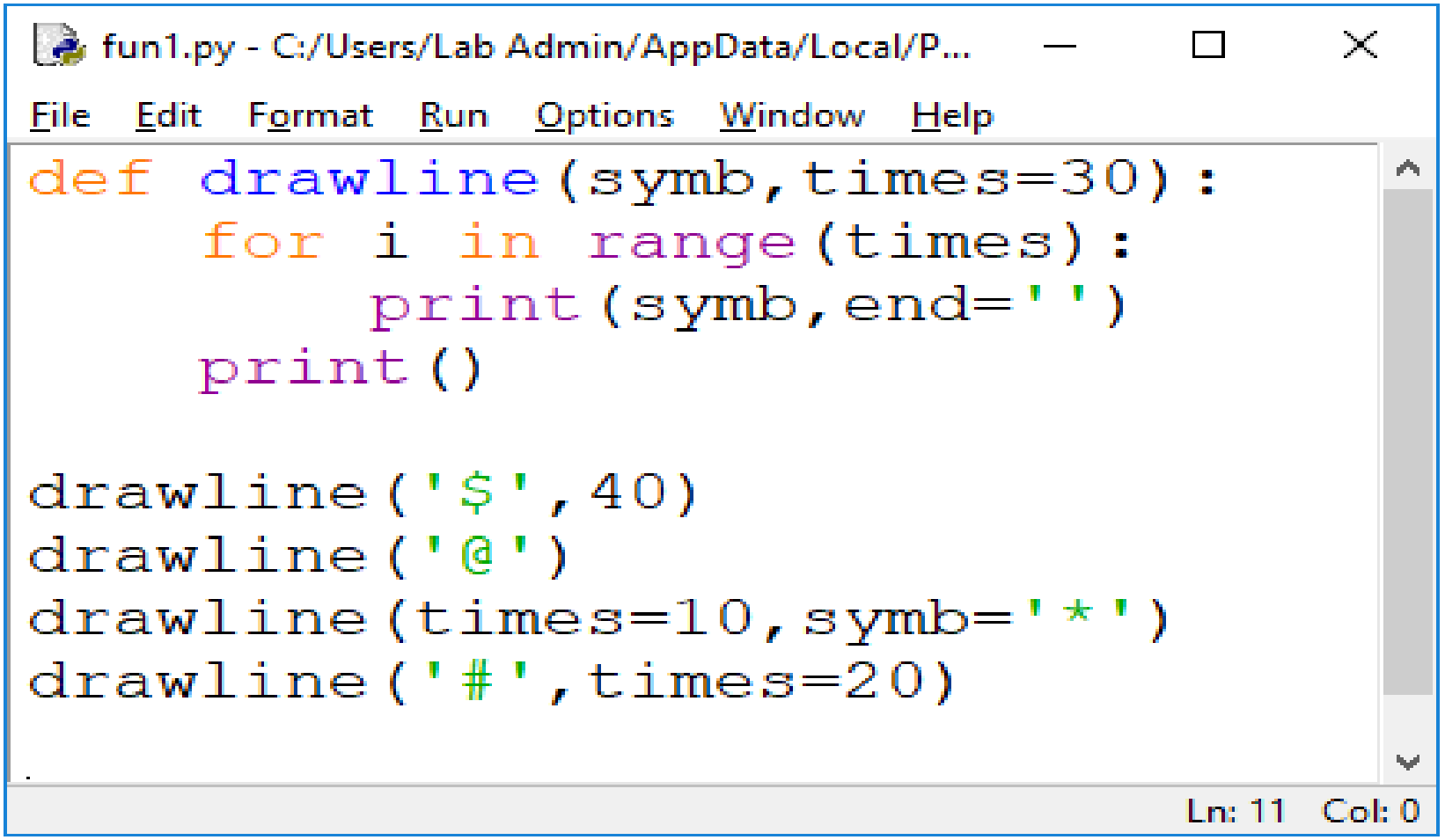
Default arguments



Keyword(Named) Arguments

- The default keyword gives flexibility to specify default value for a parameter so that it can be skipped in the function call, if needed. However, still we cannot change the order of arguments in function call i.e. you have to remember the order of the arguments and pass the value accordingly.
- To get control and flexibility over the values sent as arguments, python offers KEYWORD ARGUMENTS.
- This allows to call function with arguments in any order using name of the arguments.

Keyword(Named) Argument



Rules for combining all three type of arguments

- An argument list must first contain positional arguments followed by keyword arguments
- Keyword arguments should be taken from the required arguments
- You cannot specify a value for an argument more than once

Returning Multiple values

- Unlike other programming languages, python lets you return more than one value from function.
- The multiple return value must be either stored in TUPLE or we can UNPACK the received value by specifying the same number of variables on the left of assignment of function call.
- Let us see an example of both :-

Multiple return value stored in TUPLE

fun2.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Python36-32/fun2...

File Edit Format Run Options Window Help

```
def add10(x,y,z):
    return x+10, y+10, z+10


n1 = int(input("Enter First number"))
n2 = int(input("Enter Second number"))
n3 = int(input("Enter Third number"))
newnum = add10(n1,n2,n3)
print(newnum)
```

Ln: 9 Col: 13

```
Enter First number10
Enter Second number20
Enter Third number30
(20, 30, 40)
>>>
```

Multiple return value stored by unpacking in multiple variables

fun2.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Python36-32/fun2...

File Edit Format Run Options Window Help

```
def add10(x,y,z):
    return x+10, y+10, z+10


n1 = int(input("Enter First number"))
n2 = int(input("Enter Second number"))
n3 = int(input("Enter Third number"))
n1,n2,n3 = add10(n1,n2,n3)
print(n1)
print(n2)
print(n3)
```

Ln: 11 Col: 8

```
Enter First number11
Enter Second number22
Enter Third number33
21
32
43
>>>
```

Composition

- Refers to using an expression as a part of large expression, or a statement as a part of large statement.
- Examples
  - Max((a+b),(c+a))
  - **name="Vikram“**
  - **print(name.replace("m","nt").upper()) #function**

Scope of Variables

- SCOPE means in which part(s) of the program, a particular piece of code or data is accessible or known.
- In Python there are broadly 2 kinds of Scopes:
  - Global Scope
  - Local Scope

Global Scope

- A name declared in top level segment(main) of a program is said to have global scope and can be used in entire program.
- Variable defined outside all functions are global variables.

Local Scope

- A name declare in a function body is said to have local scope i.e. it can be used only within this function and the other block inside the function.
- The formal parameters are also having local scope.
- Let us understand with example….

Example – Local and Global Scope

fun2.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Python36-32/fun2.py (3.6.4)

File Edit Format Run Options Window Help

```
def area(length, breadth):
    a = length * breadth     # 'a' is Local to function area
    return a

l = int(input("Enter Length "))
b = int(input("Enter Breadth "))
ar = area(l,b)
# Variable l, b, ar are or global scope
print("Area of Rectangle = ",ar)
```

Ln: 3 Col: 12

Example – Local and Global Scope

fun2.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Python36-32/fun2.py (3.6.4)

File Edit Format Run Options Window Help

```
def area(length, breadth):
    a = length * breadth      # 'a' is Local to function area
    return a

l = int(input("Enter Length "))
b = int(input("Enter Breadth "))
ar = area(l,b)
# Variable l, b, ar are or global scope
print("Area of Rectangle = ",a)
```

```
Enter Length 2
Enter Breadth 4
Traceback (most recent call last):
  File "C:/Users/Lab Admin/AppData/Local/Programs/
ine 9, in <module>
    print("Area of Rectangle = ",a)
NameError: name 'a' is not defined
>>>
```

'a' is not accessible here because it is declared in function area(), so scope is local to area()

Example – Local and Global Scope

fun2.py - C:/Users/Lab Admin/AppData/Local/Programs...

File Edit Format Run Options Window Help

```
def area(length, breadth):
    a = length * breadth
    return a

def showarea():
    print("Area of Circle =",ar)

l = int(input("Enter Length "))
b = int(input("Enter Breadth "))
ar = area(l,b)
showarea()
```

Ln: 11 Col: 10

Variable "ar" is accessible in function showarea() because it is having Global Scope

```
count=0
def add(x,y):
    global count
    count+=1
    return x+y
def sub(x,y):
    global count
    count+=1
    return x-y
def mul(x,y):
    global count
    count+=1
    return x*y
def div(x,y):
    global count
    count+=1
    return x//y
ans=1
while ans==1:
    n1=int(input("Enter First Number "))
    n2=int(input("Enter Second Number "))
    op=int(input("Enter operation desired(1-Add,2-Sub,3-Mul,4-Div)"))
    if op==1:
            print("Sum=",add(n1,n2))
    elif op==2:
            print("Sub=",sub(n1,n2))
    elif op==3:
            print("Mul=",mul(n1,n2))
    elif op==4:
            print("Div=",div(n1,n2))
    ans=int(input("Do More(1-Yes) ?"))
print("Total Operation Done =",count)
```

This declaration "**global count**" is necessary for using global variables in function, other wise an error "**local variable 'count' referenced before assignment**" will appear because local scope will create variable "count" and it will be found unassigned

Lifetime of Variable

- Is the time for which a variable lives in memory. For Global variables the lifetime is entire program run i.e. as long as program is executing. For Local variables lifetime is their function's run i.e. as long as function is executing.

Name Resolution (Scope Resolution)

- For every name used within program python follows name resolution rules known as **LEGB** rule.
- **(i) LOCAL : first check whether name is in local environment, if yes Python uses its value otherwise moves to (ii)**
- **(ii) ENCLOSING ENVIRONMENT: if not in local, Python checks whether name is in Enclosing Environment, if yes Python uses its value otherwise moves to (iii)**
- **GLOBAL ENVIRONMENT: if not in above scope Python checks it in Global environment, if yes Python uses it otherwise moves to (iv)**
- **BUILT-IN ENVIRONMENT: if not in above scope, Python checks it in built-in environment, if yes, Python uses its value otherwise Python would report the error:**
- name <variable> not defined

Predict the output

ooo1.py - C:/Users/Lab Admin/AppD...

File Edit Format Run Options Window Help

```
def check():
    value=100
    print(value)

value=600
print(value)
check()
print(value)
```

Ln: 8 Col: 12

Program with variable “value” in both LOCAL and GLOBAL SCOPE

Predict the output

ooo1.py - C:/Users/Lab Admin/AppD...

File Edit Format Run Options Window Help

```
def check():
    value=100
    print(value)

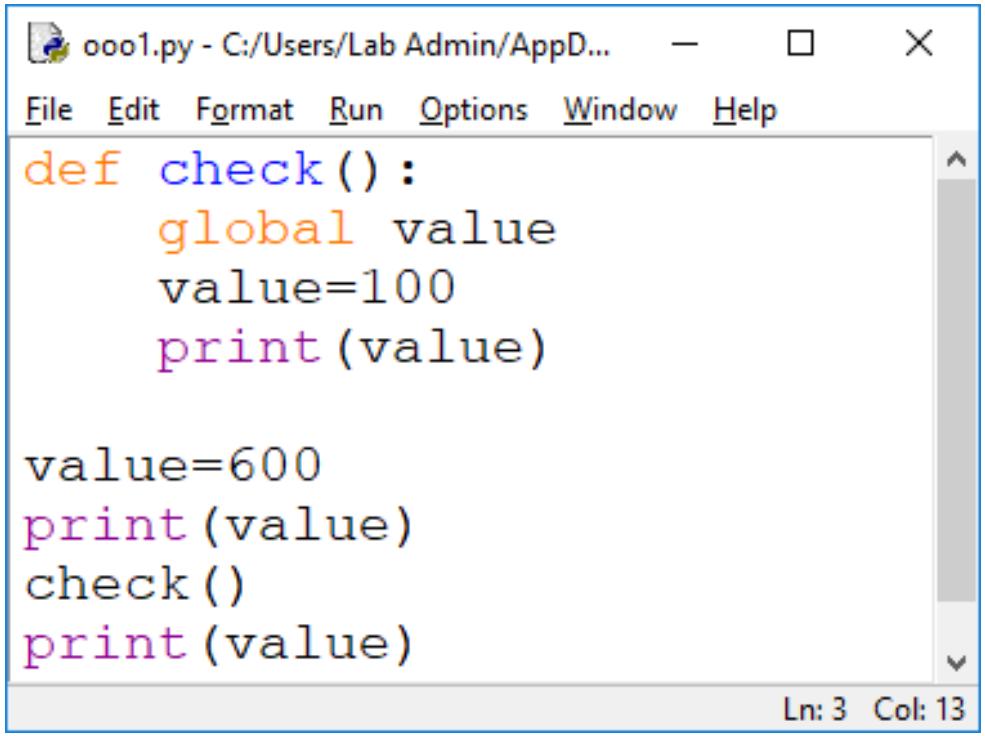
value=600
print(value)
check()
print(value)
```

Ln: 8 Col: 12

```
 RESTART: C:/Users/Lab A
600
100
600
>>>
```

Program with variable “value” in both LOCAL and GLOBAL SCOPE

Predict the output



Using GLOBAL variable "value" in local scope

Predict the output

ooo1.py - C:/Users/Lab Admin/AppD...

File Edit Format Run Options Window Help

```
def check():
    global value
    value=100
    print(value)

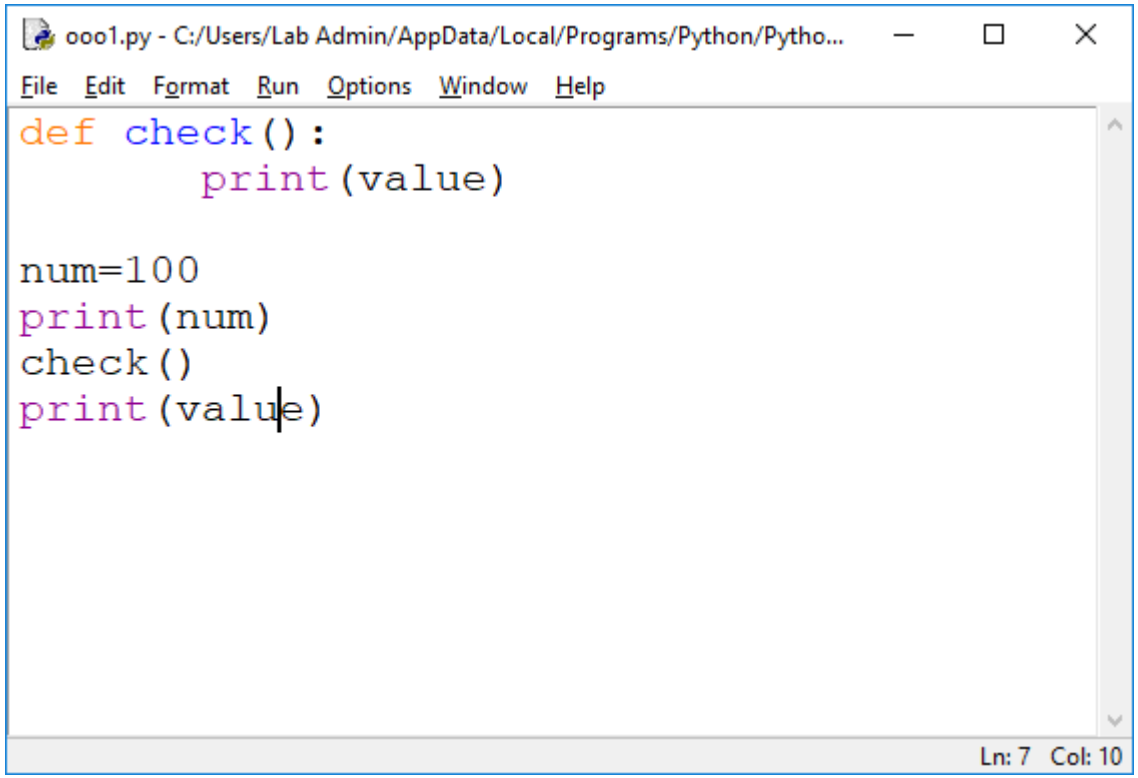
value=600
print(value)
check()
print(value)
```

Ln: 3 Col: 13

```
 RESTART: C:/Users/L
600
100
100
>>>
```

Using GLOBAL variable “value” in local scope

Predict the output



Variable “value” neither in local nor global scope

Predict the output

ooo1.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Pytho...

File Edit Format Run Options Window Help

```
def check():
        print(value)

num=100
print(num)
check()
print(value)
```

```
Traceback (most recent call last):
  File "C:/Users/Lab Admin/AppData/Local/Prog
ine 6, in <module>
    check()
  File "C:/Users/Lab Admin/AppData/Local/Prog
ine 2, in check
    print(value)
NameError: name 'value' is not defined
>>>
```

Variable "value" neither in local nor global scope

Predict the output

ooo1.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Pytho...

File Edit Format Run Options Window Help

```
def check():
        if num1>num2:
            print(num1)
        else:
            print(num2)

num1=int(input("Enter first number "))
num2=int(input("Enter Second number "))
check()
```

Ln: 7 Col: 26

Variable in Global not in Local (input in variable at global scope)

Predict the output

ooo1.py - C:/Users/Lab Admin/AppData/Local/Programs/Python/Pytho...

File Edit Format Run Options Window Help

```
def check():
        if num1>num2:
            print(num1)
        else:
            print(num2)

num1=int(input("Enter first number "))
num2=int(input("Enter Second number "))
check()
```

```
 RESTART: C:/Users/Lab Ad
Enter first number 20
Enter Second number 10
20
>>>
```

Variable in Global not in Local (input in variable at global scope)